Evaluation Report

Group 11

1 Feature engineering analysis

Selected features for this specific dataset and the assumption of why it affects the tip amount:

- `passenger_count` (more passengers tend to result in higher tips)
- `trip_distance` (larger distance might result in higher tips)
- `fare_amount` (higher fare might give lower tip)
- `extra` (extra charges can have a negative effect)
- `mta_tax` (taxes can also have a negative effect)
- `tolls_amount` (tolls can also be discouraging in giving higher tips)
- `improvement_surcharge` (a fare-related surcharge that may provide limited information about the trip)
- `airport_fee` (trips to the airport might get higher tips)
- Extreme weather conditions might result in higher tips, so we included the features describing the weather at pickup and drop-off
  - `temp_pu`
  - `rhum_pu`
  - `prcp_pu`
  - `wspd_pu`
  - `pres_pu`
  - `cldc_pu`
  - `coco_pu`
  - `temp_do`
  - `rhum_do`
  - `prcp_do`
  - `wspd_do`
  - `pres_do`
  - `cldc_do`
  - `coco_do`
- Extreme air quality values can have a positive effect on the tip, so we included the pickup and drop-off air quality averages
  - `average_value_pu`
  - `average_value_do`
- `vendor_id` (different vendors have different services and might result in different tips)
- `rate_code_id` (different rates might affect the tip amount)
- People in different parts of NY might give different amounts, so we included the features describing the pickup and drop-off locations

  - `dropoff_borough`
  - `dropoff_zone_name`
  - `dropoff_service_zone`
  - `pickup_borough`
  - `pickup_zone_name`
  - `pickup_service_zone`
- Depending on the time of the trip, during extreme hours, the tip might be higher
  - `pickup_datetime`
  - `dropoff_datetime`

Datetime features required the most preprocessing. Each datetime column is expanded into month, day, weekend, and cyclic sine/cosine representations of the month, day of the year, and hour. The extracted month and day values are also encoded as categorical features before all derived values are added to the final feature vector.

The taxi_trips dataset contributed the most valuable features because it contains the target variable and the main trip and fare attributes. The taxi-zone lookup enriches the trip data with pickup and drop-off location features. Weather and air-quality datasets provide additional contextual features, but their contribution is supplementary.

2 Model evaluation results

Following the training of the model, the pipeline, containing the feature engineering and the trained random forest, is applied to the test split. For each trip it applies the stored training-set transformations and produces a predicted tip in the column `prediction`. Predictions are compared to the true values with two metrics, *mean absolute error (MAE)* and *root mean squared error (RMSE)*, using the implementations provided by the *RegressionEvaluator* of Spark's ML-lib. MAE gives the average prediction error, while RMSE penalizes large errors more heavily. The test metrics give the final, unbiased estimate of performance on unseen data. Since we don't fine-tune the parameters of the random forest for this specific problem, the validation data remain untouched (in a different problem, we might have used the validation rows to first optimize the parameters of the prediction algorithm before evaluating it on the test set).

2.1 Prediction Results

The model trained through the platform workflow completed end-to-end training, measured through model fitting, in 89.74 seconds. On the test split, it achieved a Mean Absolute Error (MAE) of 1.2716 and a Root Mean Squared Error (RMSE) of 2.4567, both expressed in the units of the target variable. The MAE indicates that, on average, the predictions deviate from the true values by about 1.27 units. The RMSE is noticeably higher than the MAE, which shows that the error distribution contains some larger deviations, since RMSE penalizes large errors more heavily.

3 Comparison of the two data engineering approaches

To evaluate how the platform built in weeks 1-3 supports machine learning, the same training dataset was built in two different ways.

Approach A (from raw data). This approach does not reuse the structured ingestion, validation, and integration pipelines from Weeks 1–3. Starting from the four original input files (Taxi Trips, Weather, Air Quality, and Taxi Zone Lookup), a standalone Spark job manually performs source loading, schema validation, source preparation, and dataset integration. It reuses only shared utilities for reading configuration and source files, as well as the common training-set splitting function. After the raw training dataset has been generated, the same feature-engineering and model-training components are used as in Approach B.

Approach B (from the integrated platform). This approach starts from the integrated Delta tables produced in weeks 1-3. The existing ingestion, validation, and integration components are reused as they are. The new feature engineering and splitting pipeline developed in this week is added on top.

3.1 Step-by-Step Comparison

Table 1 shows which steps had to be implemented by hand in each approach.

Step	**Approach A (raw)**	**Approach B (platform)**
Loading	Read raw files and validate their schemas based on their configs	Already ingested into Delta tables
Cleaning and filtering	Invoke column operations and quality checks	Already applied by the silver layer
Integration (joins)	Re-implement joins of trips to zones, and weather and air quality by hour	Already available in the integrated tables
Splitting	Invoked directly	Invoked directly
Feature engineering	Invoked directly	Invoked directly

Table 1: Steps implemented manually in each approach.

3.2 Ensuring a Fair Comparison

After both datasets were generated, they were compared by row count, schema, and summary statistics to verify that they were equivalent. Both datasets contained 9,360,061 records.

3.3 Comparison Criteria

3.3.1 Implementation complexity

Approach A required writing and debugging every stage, including schema validations, operations, quality checks, and join logic. Approach B uses the already existing pipeline features. This means that the latter implementation has a cleaner architecture since it has different jobs for different steps (like loading/cleaning/integrating), making it more maintainable and allowing new features to be introduced more easily (it also allows many steps to be split across different machines, making it scalable).

3.3.2 Preprocessing complexity

In Approach A, data quality issues (nulls, duplicates, etc) had to be handled by new scripts (that use the existing logic), and not by the features developed during past weeks. In Approach B, these problems were already resolved, since the ML pipeline reads directly from the gold layer and could assume clean and consistent inputs and focus on feature construction.

3.3.3 Training time

Both approaches use the same feature-engineering and model-training components. The measured time includes dataset construction, feature preprocessing, pipeline fitting, and random-forest fitting. Approach B reduced this end-to-end time from 203.11 seconds to 89.74 seconds, a reduction of about 56% (approximately 2.3 times faster), because it reads the prepared Gold table instead of repeating source loading, validation, preparation, and integration.

3.3.4 Reproducibility

Approach B is more reproducible because the integrated tables are versioned delta tables, so the exact input to a training run can be recovered through time travel, and the cleaning and joining logic is defined once instead of being copied between scripts. Approach A depends on raw input files and separately maintained validation and integration logic. It also creates temporary trip IDs during execution, so the same records are not guaranteed to be assigned to the same splits across runs.

3.4 Discussion

3.4.1 Which engineering decisions from Weeks 1-3 simplified machine learning?

Organizing the platform as delta tables in a medallion architecture simplified machine learning in three ways. First, the integrated data is computed once and stored, so the ML pipeline reuses it instead of re-running the loading, cleaning, and joining steps each time. Second, the bronze and silver layers provide a source of truth for both the unprocessed and the processed data, so raw inputs are preserved and the ML pipeline reads only from the processed layers. Third, the quality checks and operations applied between layers (schema enforcement, type casting, removal of invalid records) guarantee that the processed data has

an expected format, so feature engineering does not need defensive code for malformed inputs.

3.4.2 Which datasets contributed the most useful features?

The trip data contributed the most useful features, like the trip distance and time features, which are believed to give good prediction power on our model.

3.4.3 Which parts of the workflow became reusable?

The ingestion, validation, and integration components are reusable for any future task that needs the same data. The ML pipeline itself is also reusable: the train/validation/test construction, feature engineering pipeline, and prediction logic can be applied to other prediction problems by changing only configuration inputs.

3.4.4 How did the integrated platform reduce preprocessing before feature engineering?

By the time the ML pipeline starts, the data is already loaded, typed, validated, and joined. Feature engineering therefore begins from a clean table rather than from four raw files, which removes the schema handling, quality filtering, and join alignment steps listed in Table 1. After that, the only feature engineering required is problem-specific, such as the validation of the target variable, filtering logic and feature standardization. All of these steps can differ between problems, so they could not be included in the pipelines of weeks 1-3.

3.4.5 How would the ML pipeline change if additional datasets were introduced?

As already stated, if a new dataset were added to the platform, no changes are required in the ML pipeline. The new features, target definition, filtering logic and target validation logic can be defined directly in the JSON file, and all modules act the same way.

3.4.6 What platform improvements would most help future ML applications?

A valuable improvement would be to extend the existing JSON configuration so that the user can also specify the model and its parameters, as well as the evaluation metrics used to assess the predictions. At the moment, the configuration includes data-preparation settings and some random-forest settings, but the model type and evaluation metrics are fixed in code. Letting the user define these choices in the same JSON file would make the whole workflow, from the training dataset to the evaluation, configurable and reproducible from a single place. In addition, the platform should allow the user to configure options for fine-tuning the model parameters (for example, the parameter ranges to search and the search strategy), so that hyperparameter tuning can be run automatically without changing the pipeline code.